

UNIT 1: OVERVIEW OF OPERATING SYSTEM

Course Pattern: TE IT - 2024 Pattern (SPPU)	Academic Term: Semester V (2026)
Status: Publication-Grade Theory Guide	Course Code: PCC-301-ITT

Syllabus Mapping & Unit Structure

The following table outlines the exam-oriented curriculum mapping and priority weightage for Unit 1.

Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
OS Objectives & Functions	Dual role (User interface vs Resource Manager), 3 core objectives, Dual-mode execution (Kernel/User mode), System calls, Trap instructions.	High Priority (6-8 Marks Qs)
Evolution of Operating Systems	Serial Processing, Simple Batch Systems (Resident Monitor, JCL), Multiprogrammed Batch Systems, Time-Sharing Systems (Time slicing).	High Priority (8-10 Marks Qs)
Modern OS Developments	Microkernel architecture vs Monolithic, Symmetric Multiprocessing (SMP), Multithreading, Distributed Systems, RTOS, Object-Oriented OS design.	Medium Priority (5-6 Marks Qs)
Virtual Machines & Virtualization	Virtualization concepts, Hypervisors (Type 1 Bare-Metal vs Type 2 Hosted), Full Virtualization, Para-virtualization, Containerization, VT-x / AMD-V.	High Priority (8-10 Marks Qs)
BASH Shell Scripting	Shell architecture, Core Linux commands, Permissions (chmod/chown), Process management, I/O Redirection & Piping, Shell scripting syntax, Control structures, Annotated scripts.	High Priority (8-10 Marks Qs)

Operating System Objectives and Functions

Definition and Dual Role of an Operating System

An **Operating System (OS)** is an essential layer of system software that manages computer hardware resources, provides common execution services for application programs, and acts as an authoritative intermediary between physical hardware components and system users.

From an architectural perspective, an OS operates in two fundamental capacities:

- User / Computer Interface (Hardware Abstraction Layer):**
 - Raw hardware primitives (CPU instruction sets, disk sector addresses, interrupt lines, memory controller registers) are excessively complex for application developers to program directly.
 - The OS abstracts raw physical hardware into high-level, clean conceptual models such as **processes, virtual memory address spaces, files, sockets, and device streams**.

- System functionality is exposed to applications through standardized software interfaces: **System Calls**, **Command Line Interfaces (CLI)**, and **Graphical User Interfaces (GUI)**.

2. System Resource Manager:

- A computer system comprises finite physical resources: Processor cores (CPU), Main Memory (RAM), Secondary Storage (SSDs/HDDs), and Input/Output (I/O) peripheral devices.
- The OS manages resource allocation in a controlled, safe, and multiplexed manner across competing applications via two allocation strategies:
 - **Time Multiplexing:** Programs take sequential turns utilizing a shared physical resource over time intervals (e.g., CPU thread scheduling, printer queue management).
 - **Space Multiplexing:** Multiple programs concurrently receive a dedicated portion of a physical resource (e.g., RAM memory partitioning, disk block allocation).

Three Core Operational Objectives

An Operating System is engineered to satisfy three primary operational objectives:

1. **Convenience:** Hides the low-level physical complexity of computer hardware behind intuitive abstractions. Programmers write high-level applications without managing raw hardware registers or disk head movements.
2. **Efficiency:** Maximizes hardware utilization (CPU processing cycles, RAM throughput, storage bandwidth) while minimizing contention and resource management overhead.
3. **Ability to Evolve (Extensibility):** Facilitates the seamless addition, testing, and introduction of new system functions, device drivers, and hardware support without disrupting existing applications.

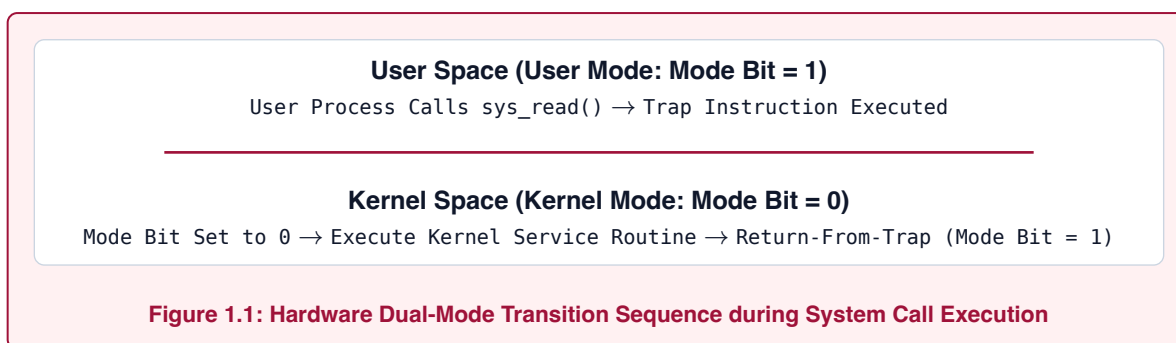
Hardware Protection and Dual-Mode Operation

To prevent errant user applications from crashing the system, corrupting physical RAM, or bypassing security controls, modern processor hardware and operating systems enforce **Dual-Mode Execution**.

Parameter	Mode Bit	Hardware & Memory Access	Instruction Privilege Set
User Mode	Mode Bit = 1	Restricted hardware access. Isolated exclusively within allocated process memory address space.	Unprivileged instructions only. Privileged actions trigger an immediate hardware trap.
Kernel Mode (Supervisor Mode)	Mode Bit = 0	Complete, unrestricted access to physical RAM, CPU registers, I/O ports, and bus controllers.	Full privileged instruction set (disable interrupts, modify page tables, initiate direct disk I/O).

Step-by-Step Hardware Mode Transition Mechanism

1. **System Call / Software Trap Initiation:** A user application requires a privileged kernel service (e.g., `read()` from disk file). It populates CPU registers with service arguments and executes a software trap instruction (e.g., `syscall` or `int 0x80`).
2. **Hardware Privilege Escalation:** The CPU hardware automatically switches the Mode Bit from 1 to 0, saves the user Program Counter (PC) and CPU registers onto the kernel stack, and jumps to the address defined in the kernel **Interrupt Vector Table**.
3. **Kernel Routine Execution:** The kernel validates argument registers, verifies caller permissions, executes the requested privileged driver or file routine, and prepares return values.
4. **Privilege De-escalation & Return:** The kernel executes a privileged return-from-trap instruction (e.g., `sysret` or `iret`). The hardware restores the Mode Bit to 1, restores user CPU registers, and resumes user application execution.



Source: Stallings, Ch 2, p. 39-45 | Source: Silberschatz, Ch 1, p. 3-25

The Evolution of Operating Systems

Operating systems have evolved through distinct technological generations driven by hardware advancements and the continuous demand for higher processing efficiency.

1. Serial Processing (1940s – 1950s)

In early electronic computers (vacuum tubes and mainframes), no Operating System existed. Programmers interacted directly with bare physical hardware.

1. **Direct Console Access:** Users reserved computer processing time in fixed blocks using physical sign-up logs.
2. **Manual Setup Procedures:** Running a program required manually loading the compiler tape, source code punch cards, assembling object code, linking libraries, and mounting output paper tapes.
3. **Core Limitations:**
 - **Scheduling Inefficiency:** Reserved machine time was wasted if a program finished early or aborted immediately due to a syntax error.
 - **Excessive Setup Time:** Machine time was predominantly consumed setting up physical paper tapes and toggle switches rather than executing code.

2. Simple Batch Systems (1950s – 1960s)

To eliminate idle setup time, manufacturers introduced the **Resident Monitor** (the earliest OS kernel predecessor).

1. **Batch Processing Workflow:** Operators collected jobs from programmers, batched similar jobs onto magnetic tapes, and submitted the tape to the resident monitor.
2. **Automated Job Sequencing:** The resident monitor remained permanently loaded in lower physical RAM. Upon job completion, it automatically loaded and executed the next job without operator intervention.
3. **Key Technical Controls:**
 - **Job Control Language (JCL):** Special control cards specifying job requirements to the monitor (e.g., `$JOB`, `$FORT`, `$RUN`, `$END`).
 - **Memory Protection:** Hardware base registers prevented running user jobs from overwriting the resident monitor memory space.
 - **Timer Interrupts:** Hardware timers prevented a single job from monopolizing the CPU in an infinite loop.

3. Multiprogrammed Batch Systems (1960s – 1970s)

In simple batch systems, the CPU remained severely underutilized (80% – 90% idle time) because I/O devices (card readers, tape drives) were orders of magnitude slower than electronic CPU clock speeds.

1. **Multiprogramming Principle:** Multiple jobs are loaded simultaneously into distinct partitions of physical RAM.
2. **Interleaved Execution:** When the active job pauses to wait for an I/O operation to complete, the OS context-switches the CPU to execute another memory-resident job.

3. **Efficiency Gain:** Increases CPU utilization from 15% – 20% in uniprogramming up to 80% – 95% in multiprogramming.

4. Time-Sharing Systems (1970s – Present)

While multiprogrammed batch systems maximized CPU utilization, they lacked interactive user feedback. **Time-Sharing (Multitasking)** was engineered to provide interactive multi-user access.

1. **Time Slicing / Quantum:** The OS assigns each user process a fixed execution time slice (10 – 50 ms) via hardware clock interrupts.
2. **Preemptive Context Switching:** When a process's quantum expires, the OS preempts it, saves register state, and switches execution to the next user process in the ready queue.
3. **Interactive Illusion:** Rapid millisecond switching creates the illusion that each user possesses a dedicated computer system.

Parameter-Based Comparison of OS Evolutionary Eras

Parameter	Serial Processing	Simple Batch	Multiprogrammed	Time-Sharing
Primary Goal	Direct hardware access.	Minimize setup time.	Maximize CPU utilization.	Minimize response time.
Memory Jobs	1 job in RAM.	1 job + Resident Monitor.	Multiple jobs in RAM.	Multiple interactive jobs.
CPU Switch	None (Manual).	Sequential on completion.	Non-preemptive on I/O wait.	Preemptive on Time Quantum.
User Interface	Hardware switches/tapes.	Punch cards & JCL.	Batch job submissions.	Interactive CLI/Terminals.
CPU Efficiency	Extremely Low (< 5%).	Low (15% – 20%).	High (80% – 95%).	Optimized for Response Time.

Source: Stallings, Ch 2, p. 46-58 | Source: Deitel, Ch 1, p. 34-48

Developments Leading to Modern Operating Systems

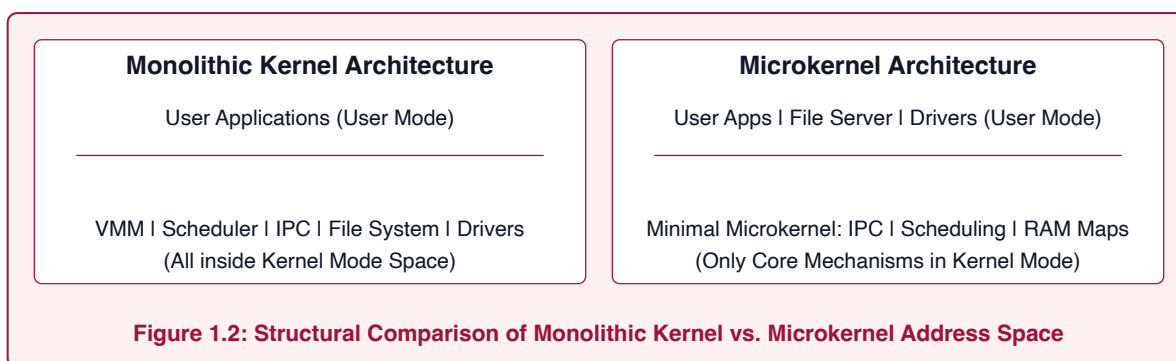
1. Monolithic vs. Microkernel Architecture

1. Monolithic Kernel Architecture:

- All OS core services (File Systems, Virtual Memory Manager, CPU Scheduler, Device Drivers, Network Stacks) execute within a single unified address space in **Kernel Mode**.
- **Advantage:** Maximum execution speed due to direct kernel function calls without IPC overhead.
- **Disadvantage:** Poor fault isolation—a single bug in a third-party device driver can crash the entire operating system kernel (**Kernel Panic / BSOD**).

2. Microkernel Architecture:

- Strips down kernel mode code to the absolute minimal required mechanisms: **low-level address space management**, **thread scheduling primitives**, and **Inter-Process Communication (IPC)**.
- Non-essential services (File Systems, Device Drivers, Protocol Stacks) are moved outside kernel mode to execute as isolated **User-Space Servers**.
- **Advantage:** Exceptional system reliability and security (a driver crash simply restarts the user-space server without bringing down the kernel).
- **Disadvantage:** Performance overhead due to frequent IPC context switches between user servers and kernel mode.



Parameter-Based Comparison: Monolithic vs. Microkernel

Parameter	Monolithic Kernel	Microkernel
Kernel Mode Size	Large (Includes all OS subsystem services).	Minimal (Core IPC, basic memory maps, thread primitives).
Fault Isolation	Poor (Driver crash brings down entire system).	Excellent (Driver crash is isolated within user space).
Execution Speed	Faster (Direct function calls in kernel RAM).	Slower (Message-passing IPC & context switch overhead).
Extensibility	Difficult (Requires kernel compilation/modules).	Easy (Add or modify user-space server processes).
Industry Examples	Linux, Windows NT, FreeBSD, macOS (Hybrid).	Mach, L4, QNX, MINIX 3.

2. Symmetric Multiprocessing (SMP) and Real-Time Systems

1. Symmetric Multiprocessing (SMP):

- Multiple physical CPU cores share a common physical RAM address space and I/O bus.
- All processors are equal peers; any CPU core can execute user threads or kernel code concurrently.
- The OS scheduler enforces multi-core load balancing, mutual exclusion locking, and CPU cache coherence.

2. Real-Time Operating Systems (RTOS):

- Engineered for environment control systems where correctness depends on meeting strict temporal deadlines.
- **Hard Real-Time:** Missing a deadline results in catastrophic system failure (e.g., automotive airbag, flight control system).
- **Soft Real-Time:** Missing a deadline degrades service quality but does not cause complete system failure (e.g., video playback).

Source: Stallings, Ch 2, p. 59-75 | Source: Tanenbaum, Ch 1, p. 15-28

Virtual Machines and Virtualization

Fundamental Principles of Virtualization

Virtualization abstracts physical hardware (CPU, RAM, Disks, NICs) into software representations known as **Virtual Machines (VMs)**.

1. **Host System:** The underlying physical server hardware and host operating system.
2. **Guest System:** The virtualized guest operating system executing inside an isolated VM environment.

3. **Virtual Machine Monitor (VMM) / Hypervisor:** The core virtualization software layer that intercepts privileged guest instructions, manages hardware resource allocation, and enforces VM boundary isolation.

Hypervisor Classification: Type 1 vs. Type 2

Parameter	Type 1 Hypervisor (Bare-Metal)	Type 2 Hypervisor (Hosted)
Architecture Level	Runs directly on bare physical server hardware. No host OS underneath.	Runs as an application layer on top of an existing host operating system.
Overhead & Speed	Minimal virtualization overhead; near-native hardware speed.	Higher latency due to host OS abstraction and device driver layers.
Resource Management	Directly schedules hardware CPU cores and physical memory pages.	Relies on host OS process scheduler for physical hardware access.
Target Environment	Enterprise cloud data centers (AWS, Azure, private clouds).	Desktop development, software testing, cross-platform emulation.
Examples	VMware ESXi, KVM, Xen, Microsoft Hyper-V.	Oracle VirtualBox, VMware Workstation, Parallels.

Virtualization Approaches

1. **Full Virtualization:** The hypervisor provides a complete software emulation of underlying physical hardware. The Guest OS runs **unmodified** using **Binary Translation** for sensitive instructions.
2. **Para-virtualization:** The Guest OS kernel is modified specifically to be aware of the hypervisor. Privileged instructions are replaced with direct API calls to the hypervisor called **Hypercalls**.
3. **OS-Level Virtualization (Containerization):** Isolates user-space instances using kernel features (**namespaces** and **cgroups**) while sharing a single host kernel (e.g., Docker, LXC).

Hardware-Assisted Virtualization (Intel VT-x / AMD-V)

Modern CPUs provide native hardware virtualization extensions to resolve trap-and-emulate issues on x86 architectures:

- Introduces two CPU execution modes: **VMX Root Operation** (Hypervisor) and **VMX Non-Root Operation** (Guest OS).
- Provides **Extended Page Tables (EPT)** / **Nested Page Tables (NPT)** for hardware-accelerated memory address translation from Guest Physical Memory → Host Physical RAM.

Source: Silberschatz, Ch 18, p. 725-735 | Source: Tanenbaum, Ch 1, p. 63-68

BASH Shell Scripting

Shell Architecture and Stream Redirection

1. **Shell Role:** A user-space command interpreter program acting as an interface between CLI users and the OS kernel (Variants: sh, bash, zsh, ksh).
2. **Standard File Descriptors:**
 - **stdin (FD 0):** Standard input stream (default: keyboard).
 - **stdout (FD 1):** Standard output stream (default: terminal display).
 - **stderr (FD 2):** Standard error stream (default: terminal display).
3. **Redirection Operators:**
 - `cmd > file.txt`: Redirects stdout to file.txt (overwrites).
 - `cmd >> file.txt`: Redirects stdout to file.txt (appends).

- `cmd 2> err.log`: Redirects stderr to err.log.
- `cmd1 | cmd2`: **Pipeline** — Connects stdout of cmd1 directly into stdin of cmd2 via kernel pipe buffer.

Core Linux Command & Permission Bit Reference

Category	Command Syntax	Operational Description
File & Directory	<code>ls -la</code> <code>mkdir -p dir</code> <code>rm -rf path</code>	List all files including hidden with permissions. Create directory hierarchy. Forcefully remove files/directories recursively.
Permissions	<code>chmod 755 script.sh</code> <code>chmod u+x file</code> <code>chown user:grp file</code>	Set octal permissions (rwxr-xr-x). Grant execute permission to owner. Change file owner user and group.
Process Control	<code>ps aux</code> <code>top</code> <code>kill -9 PID</code> <code>cmd &</code>	Display all running processes with CPU/RAM. Interactive real-time process manager. Send SIGKILL signal to forcefully terminate PID. Run command in background asynchronously.

Octal Permission Bit Calculation

Permissions are divided into 3 triads: **Owner (u)**, **Group (g)**, **Others (o)**.

- Read ($r = 4$), Write ($w = 2$), Execute ($x = 1$).
- Example 755: Owner = $4 + 2 + 1 = 7$ (rwx), Group = $4 + 0 + 1 = 5$ (r-x), Others = $4 + 0 + 1 = 5$ (r-x).

BASH Script Programming Syntax

```
#!/bin/bash
# Shebang line: Specifies execution interpreter

# 1. Variable Assignment & Command Substitution
TARGET_DIR="./backup"
CURRENT_DATE=$(date +%Y%m%d)

# 2. Argument Checking & Positional Parameters
if [ $# -ne 1 ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

FILE_NAME="$1"

# 3. File Test & Branching Logic
if [ -f "$FILE_NAME" ] && [ -r "$FILE_NAME" ]; then
    mkdir -p "$TARGET_DIR"
    cp "$FILE_NAME" "$TARGET_DIR/${FILE_NAME}_${CURRENT_DATE}.bak"
    echo "[SUCCESS] File backed up successfully. Exit status: $?"
else
    echo "[ERROR] File does not exist or lacks read permission."
    exit 2
fi
```

Source: Mendel Cooper, ABS Guide, p. 1-50 | Source: Deitel, Ch 3, p. 124-137